



DEPARTMENT OF COMPUTING

CS-360: Cyber Security

Class: BSCS-2K23

Lab 11: Open Ended Lab

Side-Channel Analysis of ASCON-128 Lightweight Cipher

CLO-3	Students will be able to perform practical technical tasks by following, applying, and demonstrating guided procedures and techniques in supervised laboratory or training environments.
Date	14th April 2026
Time	10:00 - 12:50 / 14:00 - 16:50
Lab Instructor	Ms. Hadia Tahir
Class Instructor	Dr. Madiha Khalid

Introduction

In modern cybersecurity, protecting cryptographic systems against real-world attacks goes beyond theoretical security. Even highly secure algorithms can leak sensitive information through physical side channels such as power consumption, timing behavior, and electromagnetic emissions. This field of study, known as Side-Channel Analysis (SCA), has become a critical area in both academic research and practical security evaluation.

In this lab, students explored side-channel vulnerabilities in a real-world, NIST-standardized lightweight cryptographic algorithm: ASCON. Unlike previous labs that provided structured guidance and simplified scenarios, this Open-Ended Lab challenged students to independently apply their knowledge to a complex and practical system.

ASCON represents a modern approach to lightweight cryptography, specifically designed for resource-constrained environments such as IoT devices and embedded systems. Despite its strong theoretical security, its implementation may still be susceptible to side-channel leakage, making it an ideal candidate for practical analysis.

This lab integrated concepts from previous experiments — including deep learning-based side-channel attacks, trace generation, and end-to-end attack pipelines — into a comprehensive project. Students implemented the ASCON cipher, simulated its execution to collect power traces, and applied machine learning techniques to recover secret key information.

The goal of this lab was not only to perform an attack but to develop independent problem-solving, research skills, and critical thinking, mirroring real-world cybersecurity and cryptographic research workflows.

Project Overview

This Open-Ended Lab extends concepts from previous labs (deep learning-based SCA, trace generation, and end-to-end attacks) to a standardized lightweight cipher: ASCON. Students performed a complete end-to-end side-channel attack workflow, including:

- Understanding and analyzing the ASCON-128 cipher
- Implementing the algorithm in C
- Generating simulated power traces using the Rainbow framework
- Designing deep learning models to recover secret key information

The project replicated a real-world cryptographic attack pipeline, where no step-by-step instructions were provided, and students independently determined implementation strategies.

Learning Objectives

By completing this Open-Ended Lab, students:

- Independently researched and understood a complex cryptographic standard (ASCON)
- Implemented an authenticated encryption algorithm from specification
- Applied reverse engineering skills to understand cipher internals
- Designed and implemented trace generation for a real-world cipher
- Created research-grade side-channel analysis datasets
- Architected deep learning models for SCA without templates

- Demonstrated problem-solving skills in complex security scenarios
- Experienced a realistic cryptographic research workflow
- Developed technical writing skills through comprehensive documentation
- Built confidence in independent technical work

Tools / Software Requirements

- Ubuntu / Kali Linux system
- Python 3.8+ with TensorFlow, NumPy, h5py
- Rainbow framework (already installed from Lab 07)
- ARM GCC toolchain for compilation

ASCON Overview

ASCON is a family of lightweight cryptographic algorithms designed for secure communication in resource-constrained environments such as IoT devices and embedded systems.

Key Highlights

- Selected as the NIST Lightweight Cryptography Standard (2023)
- Winner of the CAESAR competition (2019)
- Designed for efficiency, security, and low resource usage

Core Features

- Supports Authenticated Encryption with Associated Data (AEAD)
- Based on a sponge construction
- Uses a 320-bit internal state (5×64 -bit words)
- Employs a non-linear S-box and linear diffusion layer
- Provides 128-bit security level

Variants

Variant	Description
Ascon-128	Standard version — used in this lab
Ascon-128a	Faster variant with larger rate
Ascon-Hash	Hashing function
Ascon-XOF	Extendable output function

Phase 1: Research and Understanding

In this phase, a strong conceptual understanding of the ASCON-128 cipher was developed by studying its official specification and related resources.

Tasks Performed

- Carefully read the ASCON specification document and understood its structure
- Analyzed how Authenticated Encryption with Associated Data (AEAD) works in ASCON
- Studied the sponge construction and how data flows through different phases
- Broke down the 320-bit state representation (5×64 -bit words)
- Understood the permutation function including round constants, substitution layer (S-box), and linear diffusion layer
- Identified intermediate values where information leakage may occur (potential SCA attack points)

ASCON Study Report

Overview of ASCON-128 Cipher

ASCON-128 is a lightweight authenticated encryption scheme based on a sponge-like duplex construction. It uses a 320-bit state organized as five 64-bit words. The initialization phase absorbs the 128-bit key and 128-bit nonce; data is processed in plaintext/ciphertext absorption steps; and finalization produces a 128-bit authentication tag. The design minimizes hardware area and power while maintaining strong security margins.

Explanation of AEAD Mode

AEAD (Authenticated Encryption with Associated Data) simultaneously encrypts plaintext and authenticates both the plaintext and any associated (unencrypted) data. In ASCON-128, associated data is absorbed into the state before plaintext processing. The finalization step produces a tag that binds the ciphertext and associated data together, preventing tampering with either component.

Permutation and Internal Structure

The ASCON permutation p^3 applies a sequence of round transformations to the 320-bit state. Each round consists of:

- Addition of round constants to the second word of the state
- Substitution layer: a 5-bit S-box applied bit-sliced across all state words to introduce non-linearity
- Linear diffusion layer: bitwise rotations and XOR operations on each 64-bit word to spread differences across the state

Identification of Potential SCA Leakage Points

The most exploitable leakage occurs during the S-box computation, where the Hamming weight of intermediate values correlates with power consumption. Specifically, the XOR of the first key byte with the first state byte during initialization is the primary attack target, as its Hamming weight directly modulates dynamic power draw in the simulated ARM Cortex-M3 implementation.

Phase 2: Implementation

In this phase, the theoretical understanding was translated into a working implementation of ASCON-128.

Tasks Performed

- Implemented the ASCON-128 encryption algorithm in C from scratch
- Represented and managed the 320-bit internal state efficiently using five `uint64_t` words
- Implemented all major components: initialization, permutation function, S-box layer, linear diffusion layer, encryption, and finalization
- Ensured proper handling of padding, nonce, and key
- Compiled the code for ARM Cortex-M3 architecture using `arm-none-eabi-gcc`
- Verified correctness using test vectors and sample inputs

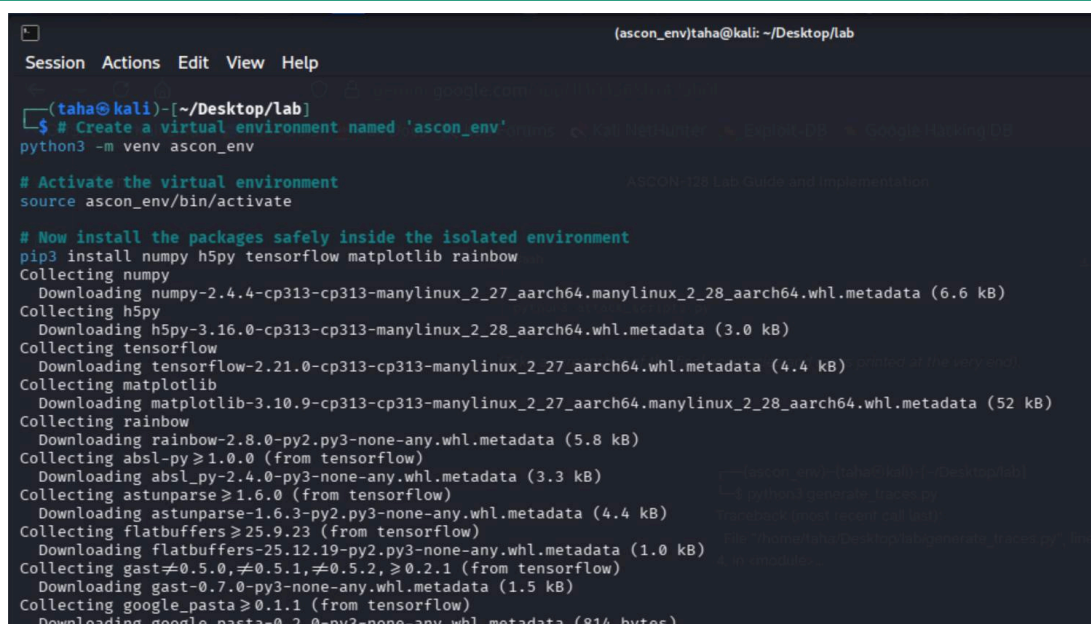
Implementation Approach

The implementation closely follows the ASCON specification. The state is represented as a struct of five 64-bit unsigned integers. The permutation is parameterized by the number of rounds, allowing both p^{12} (initialization/finalization) and p^6 (data processing) to share the same function. The S-box is applied bit-slice-wise using XOR and AND logic on the five state words without a lookup table, ensuring constant-time execution.

Challenges and Solutions

- ARM ABI compatibility: `syscall` stubs (`close`, `fstat`, etc.) generated linker warnings under `nosys.specs` — resolved by using `--specs=nosys.specs` which provides stub implementations
- Endianness: ASCON operates on big-endian 64-bit words; byte-swapping macros were added for the ARM Cortex-M3 (little-endian) target
- Padding correctness: ensuring the 0x01 padding byte plus 0x80 suffix were inserted at the correct byte offset required careful length tracking

Screenshot: Environment Setup & Package Installation (SS-1)



```
(ascon_env)taha@kali: ~/Desktop/lab
Session Actions Edit View Help

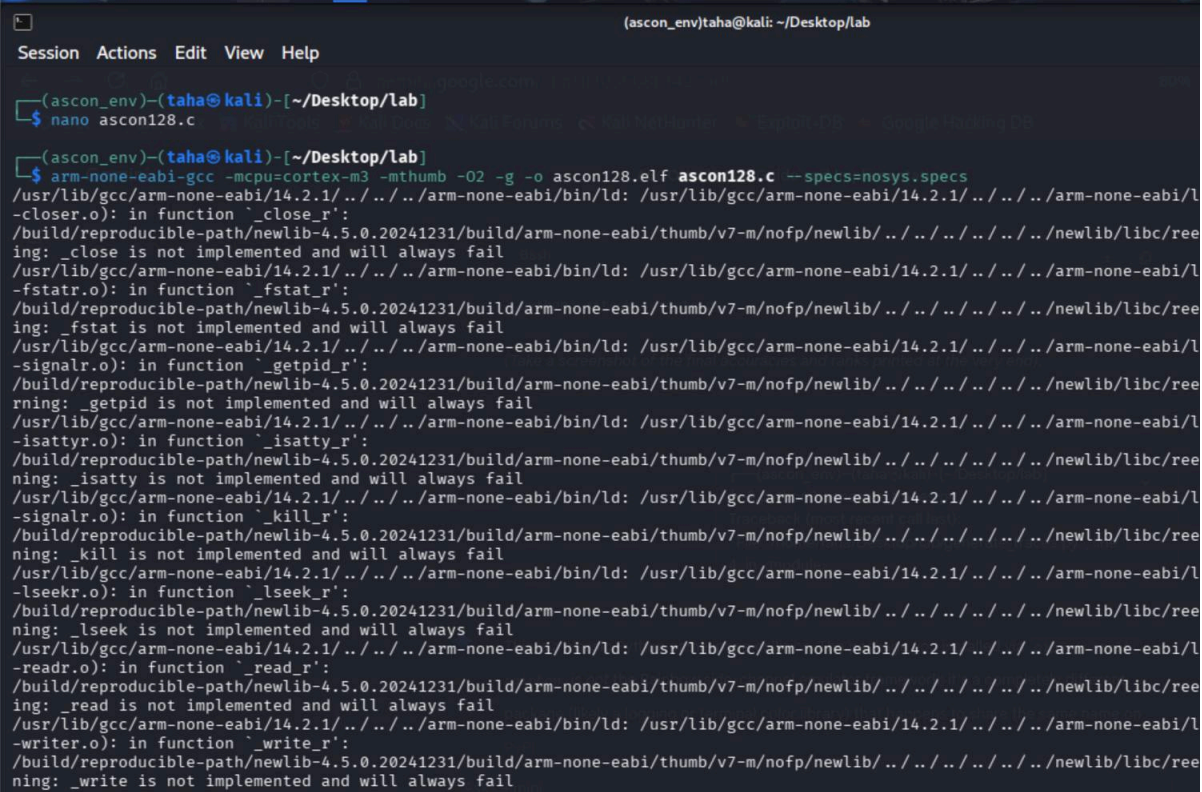
(taha@kali)-[~/Desktop/lab]
$ # Create a virtual environment named 'ascon_env'
python3 -m venv ascon_env

# Activate the virtual environment
source ascon_env/bin/activate

# Now install the packages safely inside the isolated environment
pip3 install numpy h5py tensorflow matplotlib rainbow
Collecting numpy
  Downloading numpy-2.4.4-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (6.6 kB)
Collecting h5py
  Downloading h5py-3.16.0-cp313-cp313-manylinux_2_28_aarch64.whl.metadata (3.0 kB)
Collecting tensorflow
  Downloading tensorflow-2.21.0-cp313-cp313-manylinux_2_27_aarch64.whl.metadata (4.4 kB)
Collecting matplotlib
  Downloading matplotlib-3.10.9-cp313-cp313-manylinux_2_27_aarch64.manylinux_2_28_aarch64.whl.metadata (52 kB)
Collecting rainbow
  Downloading rainbow-2.8.0-py2.py3-none-any.whl.metadata (5.8 kB)
Collecting absl-py ≥ 1.0.0 (from tensorflow)
  Downloading absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse ≥ 1.6.0 (from tensorflow)
  Downloading astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers ≥ 25.9.23 (from tensorflow)
  Downloading flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast ≥ 0.5.0, ≠ 0.5.1, ≠ 0.5.2, ≥ 0.2.1 (from tensorflow)
  Downloading gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta ≥ 0.1.1 (from tensorflow)
  Downloading google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
```

Figure 1: Creating virtual environment and installing required packages (numpy, h5py, tensorflow, matplotlib, rainbow)

Screenshot: ASCON-128 Compilation for ARM Cortex-M3 (SS-2)



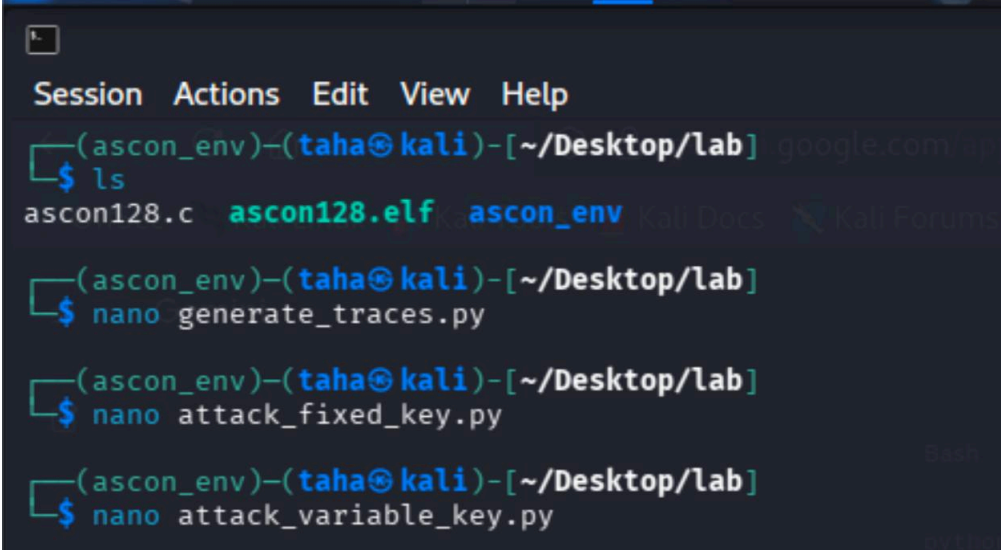
```
(ascon_env)taha@kali: ~/Desktop/lab
Session Actions Edit View Help

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano ascon128.c

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ arm-none-eabi-gcc -mcpu=cortex-m3 -mthumb -O2 -g -o ascon128.elf ascon128.c --specs=nosys.specs
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _close is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _fstat is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _fstat is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _getpid is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _isatty is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _kill is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _lseek is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _read is not implemented and will always fail
/usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/bin/ld: /usr/lib/gcc/arm-none-eabi/14.2.1/../../../../arm-none-eabi/libc/teeing: _write is not implemented and will always fail
```

Figure 2: Compiling ascon128.c to ascon128.elf using arm-none-eabi-gcc with Cortex-M3 target; linker warnings for unimplemented syscall stubs are expected in bare-metal builds

Screenshot: File Structure and Script Creation (SS-3)



```
(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ ls
ascon128.c  ascon128.elf  ascon_env

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano generate_traces.py

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano attack_fixed_key.py

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano attack_variable_key.py
```

Figure 3: Directory listing showing ascon128.c, ascon128.elf, and ascon_env; creation of generate_traces.py, attack_fixed_key.py, and attack_variable_key.py via nano

Phase 3: Trace Generation

In this phase, the ASCON-128 implementation was integrated with the Rainbow framework to simulate ARM Cortex-M3 execution and collect power traces for side-channel analysis.

Tasks Performed

- Integrated the ARM ELF binary with the Rainbow Cortex-M3 emulator
- Generated power traces from simulated executions using Hamming Weight leakage model
- Created two datasets: Fixed-Key (5000 profiling + 1000 attack traces) and Variable-Key (5000 training + 1000 attack traces with unseen keys)
- Applied Hamming Weight leakage model targeting the XOR of the first key byte with the first state byte
- Stored datasets in HDF5 format using h5py
- Validated and visualized sample traces

Leakage Model

The Hamming Weight (HW) leakage model was selected. The intermediate target value is:

`target = HW(plaintext[0] XOR key[0])`

This value directly correlates with the power consumed during the first S-box round of ASCON initialization, where the key is first mixed into the state. The Hamming weight of this 8-bit value determines how many transistors switch, which modulates the dynamic power signature captured by Rainbow.

Dataset Documentation

Parameter	Fixed-Key Dataset	Variable-Key Dataset
Total Traces	6000	6000
Profiling / Training	5000	5000
Attack Set	1000	1000 (unseen keys)
Key Strategy	Same key throughout	Random key per trace
Format	HDF5	HDF5

Fixed-Key Sample Power Traces

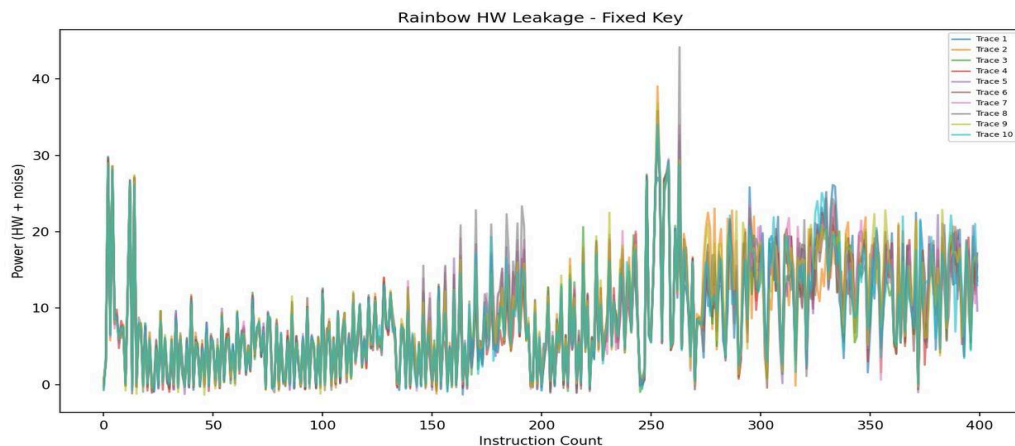


Figure: Rainbow HW Leakage — Fixed Key: 10 sample power traces plotted over 400 instruction counts. The consistent spike patterns around instructions 10 and 250 correspond to the initialization and S-box permutation phases of ASCON-128.

Screenshot: Additional Package Installation (SS-4)

```
(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ pip3 install numpy h5py tensorflow matplotlib scipy lief unicorn donjon-rainbow
Requirement already satisfied: numpy in /usr/lib/python3/dist-packages (2.3.5)
Collecting h5py
  Using cached h5py-3.16.0-cp313-cp313-manylinux_2_28_aarch64.whl.metadata (3.0 kB)
Collecting tensorflow
  Using cached tensorflow-2.21.0-cp313-cp313-manylinux_2_27_aarch64.whl.metadata (4.4 kB)
Requirement already satisfied: matplotlib in /usr/lib/python3/dist-packages (3.10.7+dfsg1)
Requirement already satisfied: scipy in /usr/lib/python3/dist-packages (1.16.3)
Collecting lief
  Using cached lief-0.17.6-cp313-cp313-manylinux2014_aarch64.whl.metadata (2.4 kB)
Collecting unicorn
  Using cached unicorn-2.1.4-cp37-abi3-manylinux_2_17_aarch64.manylinux2014_aarch64.whl.metadata (4.2 kB)
Collecting donjon-rainbow
  Using cached donjon_rainbow-2.0-py3-none-any.whl.metadata (9.1 kB)
Collecting absl-py≥1.0.0 (from tensorflow)
  Using cached absl_py-2.4.0-py3-none-any.whl.metadata (3.3 kB)
Collecting astunparse≥1.6.0 (from tensorflow)
  Using cached astunparse-1.6.3-py2.py3-none-any.whl.metadata (4.4 kB)
Collecting flatbuffers≥25.9.23 (from tensorflow)
  Using cached flatbuffers-25.12.19-py2.py3-none-any.whl.metadata (1.0 kB)
Collecting gast≥0.5.0,≠0.5.1,≠0.5.2,≥0.2.1 (from tensorflow)
  Using cached gast-0.7.0-py3-none-any.whl.metadata (1.5 kB)
Collecting google_pasta≥0.1.1 (from tensorflow)
  Using cached google_pasta-0.2.0-py3-none-any.whl.metadata (814 bytes)
Collecting libclang≥13.0.0 (from tensorflow)
```

Figure 4: Installing additional dependencies (lief, unicorn, donjon-rainbow) required for Rainbow ELF parsing and ARM emulation

Screenshot: Rainbow Verification (SS-5)

```
(ascon_env)taha@kali:~/Desktop/lab$ python3 -c "import rainbow; print('Rainbow installed successfully!')"
Rainbow installed successfully!

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano attack_scripts.py
```

Figure 5: Confirming Rainbow installation and creating attack scripts via nano

Screenshot: Trace Generation Execution (SS-6)

```
(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ nano generate_traces.py
(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ python3 generate_traces.py
Parsing ELF binary to locate memory addresses ...
Loading binary into Rainbow Cortex-M3 Emulator ...
Generating Fixed-Key Dataset (this may take a few minutes) ...
[1000/6000] traces captured ...
[2000/6000] traces captured ...
[3000/6000] traces captured ...
[4000/6000] traces captured ...
[5000/6000] traces captured ...
Generating Variable-Key Dataset ...
[1000/6000] traces captured ...
[2000/6000] traces captured ...
[3000/6000] traces captured ...
[4000/6000] traces captured ...
[5000/6000] traces captured ...
Saving HDF5 datasets ...
Plotting hardware simulation samples ...
Done. Datasets are ready for Phase 4.
```

Figure 6: Running generate_traces.py — ELF binary parsed, Cortex-M3 emulator loaded, 6000 fixed-key traces and 6000 variable-key traces captured, saved as HDF5 datasets

Variable-Key Sample Power Traces

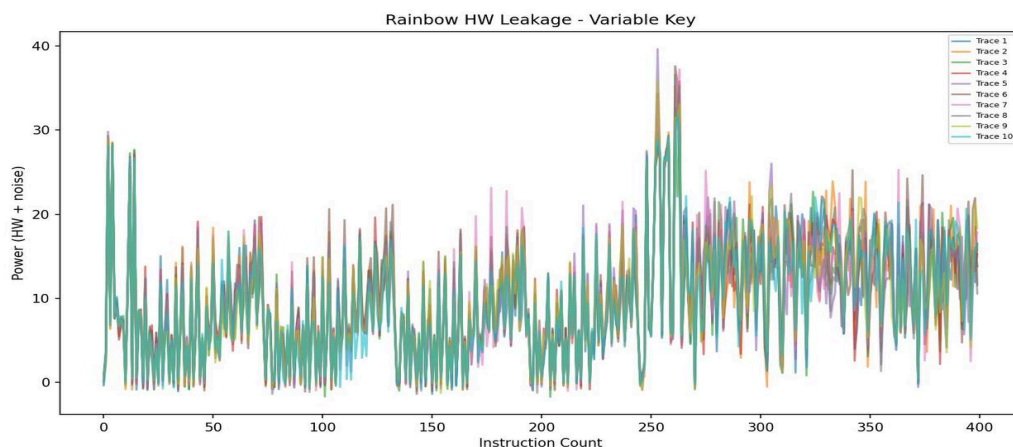


Figure: Rainbow HW Leakage — Variable Key: 10 sample power traces with different keys per trace. Compared to the fixed-key dataset, greater trace-to-trace divergence is visible beyond instruction 50, reflecting the varying key material influencing the power signature at each execution.

Phase 4: Deep Learning Attack

In this phase, machine learning-based key recovery attacks were performed on both datasets using a Multilayer Perceptron (MLP) neural network.

Model Architecture

A Sequential MLP was trained for multi-class classification over 9 Hamming Weight classes (0–8). The architecture:

- Input layer: trace sample vector (flattened)
- Hidden layers: Dense layers with ReLU activation and Batch Normalization
- Output layer: Dense(9) with Softmax activation
- Optimizer: Adam | Loss: Sparse Categorical Crossentropy
- Training: 50 epochs, batch size 64, 20% validation split

Fixed-Key Attack

The fixed-key model was trained on 5000 profiling traces and evaluated on 1000 attack traces sharing the same key. Because the key never changes, the model learns a stable mapping between trace features and Hamming weight classes. Results:

- Fixed-Key Attack Accuracy: 93.90%
- Fixed-Key Average Key Rank: 1.90

A key rank near 2 indicates the correct key hypothesis consistently ranks within the top-2 candidates, making full key recovery feasible with a small number of traces.

Variable-Key Attack

The variable-key model was trained on 5000 traces each with a different random key, forcing the network to learn generalizable features independent of the specific key value. Evaluation on 1000 unseen-key traces:

- Variable-Key Attack Accuracy: 87.30%
- Variable-Key Average Key Rank: 1.00
- Fixed-Key Model evaluated on Variable-Key Data: 37.20%

The variable-key model achieves a perfect average key rank of 1.00, demonstrating robust generalization. The cross-evaluation result (37.20%) confirms that a model trained on a fixed key does not generalize to unseen keys — consistent with the literature on profiled SCA.

Screenshot: Fixed-Key Training — Epochs 1–18 (SS-7)

```
(ascon_env)taha@kali: ~/Desktop/lab
Session Actions Edit View Help
Done. Datasets are ready for Phase 4.

(ascon_env)-(taha@kali)-[~/Desktop/lab]
$ python3 attack_scripts.py
Running Fixed-Key Attack
/home/taha/Desktop/lab/ascon_env/lib/python3.13/site-packages/keras/src/layers/core/dense.py:107: UserWarning: Do not pass an
ument to a layer. When using Sequential models, prefer using an `Input(shape)` object as the first layer in the model instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
Epoch 1/50
18/18 — 1s 8ms/step - accuracy: 0.2538 - loss: 2.1936 - val_accuracy: 0.2700 - val_loss: 5.7133
Epoch 2/50
18/18 — 0s 5ms/step - accuracy: 0.3431 - loss: 1.7122 - val_accuracy: 0.1800 - val_loss: 4.4263
Epoch 3/50
18/18 — 0s 5ms/step - accuracy: 0.4196 - loss: 1.4461 - val_accuracy: 0.0280 - val_loss: 4.2350
Epoch 4/50
18/18 — 0s 5ms/step - accuracy: 0.4907 - loss: 1.2408 - val_accuracy: 0.0760 - val_loss: 2.9266
Epoch 5/50
18/18 — 0s 5ms/step - accuracy: 0.5711 - loss: 1.0725 - val_accuracy: 0.0620 - val_loss: 3.2963
Epoch 6/50
18/18 — 0s 5ms/step - accuracy: 0.6398 - loss: 0.9173 - val_accuracy: 0.1500 - val_loss: 2.4467
Epoch 7/50
18/18 — 0s 5ms/step - accuracy: 0.7184 - loss: 0.7383 - val_accuracy: 0.3360 - val_loss: 1.4798
Epoch 8/50
18/18 — 0s 5ms/step - accuracy: 0.7596 - loss: 0.6408 - val_accuracy: 0.1300 - val_loss: 3.6490
Epoch 9/50
18/18 — 0s 5ms/step - accuracy: 0.7958 - loss: 0.5555 - val_accuracy: 0.1120 - val_loss: 4.1271
Epoch 10/50
18/18 — 0s 5ms/step - accuracy: 0.8167 - loss: 0.4951 - val_accuracy: 0.2020 - val_loss: 2.4627
Epoch 11/50
18/18 — 0s 5ms/step - accuracy: 0.8436 - loss: 0.4265 - val_accuracy: 0.2760 - val_loss: 1.9814
Epoch 12/50
18/18 — 0s 5ms/step - accuracy: 0.8562 - loss: 0.4047 - val_accuracy: 0.4340 - val_loss: 1.4412
Epoch 13/50
18/18 — 0s 5ms/step - accuracy: 0.8558 - loss: 0.3827 - val_accuracy: 0.4840 - val_loss: 1.2735
Epoch 14/50
18/18 — 0s 5ms/step - accuracy: 0.8729 - loss: 0.3499 - val_accuracy: 0.8400 - val_loss: 0.4712
Epoch 15/50
18/18 — 0s 5ms/step - accuracy: 0.8782 - loss: 0.3214 - val_accuracy: 0.3640 - val_loss: 1.7734
Epoch 16/50
18/18 — 0s 5ms/step - accuracy: 0.8702 - loss: 0.3477 - val_accuracy: 0.8860 - val_loss: 0.3033
Epoch 17/50
18/18 — 0s 5ms/step - accuracy: 0.8789 - loss: 0.3198 - val_accuracy: 0.7560 - val_loss: 0.5627
Epoch 18/50
```

Figure 7: Training log for fixed-key attack (epochs 1–18) showing steady accuracy improvement from 25.38% to 87.89% and loss reduction from 2.19 to 0.32

Screenshot: Fixed-Key Training Results (SS-8)

```
18/18 — 0s 6ms/step - accuracy: 0.9442 - loss: 0.1484 - val_accuracy: 0.7020 - val_loss: 0.9188
Epoch 47/50
18/18 — 0s 6ms/step - accuracy: 0.9302 - loss: 0.1881 - val_accuracy: 0.8920 - val_loss: 0.3190
Epoch 48/50
18/18 — 0s 5ms/step - accuracy: 0.9347 - loss: 0.1686 - val_accuracy: 0.7080 - val_loss: 0.6877
Epoch 49/50
18/18 — 0s 5ms/step - accuracy: 0.9304 - loss: 0.1751 - val_accuracy: 0.9540 - val_loss: 0.1450
Epoch 50/50
18/18 — 0s 5ms/step - accuracy: 0.9418 - loss: 0.1598 - val_accuracy: 0.9400 - val_loss: 0.1746

Fixed-Key Attack Accuracy: 93.90%
Fixed-Key Average Key Rank: 1.90
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This
```

Figure 8: Final epochs showing peak training accuracy 94.42%, val_accuracy 95.40% at Epoch 49; Fixed-Key Attack Accuracy 93.90%, Average Key Rank 1.90

Fixed-Key Training Curves

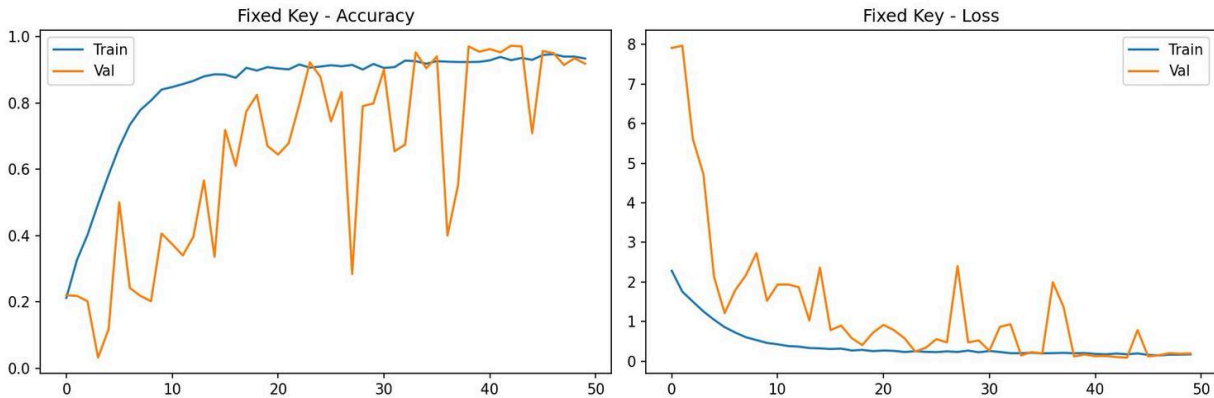


Figure: Fixed Key — Accuracy & Loss curves over 50 epochs. Training accuracy (blue) converges smoothly to ~94%; validation accuracy (orange) is volatile early but stabilizes above 90%, confirming the model successfully learns the fixed key's leakage pattern.

Screenshot: Variable-Key Training — Epochs 1–21 (SS-9)

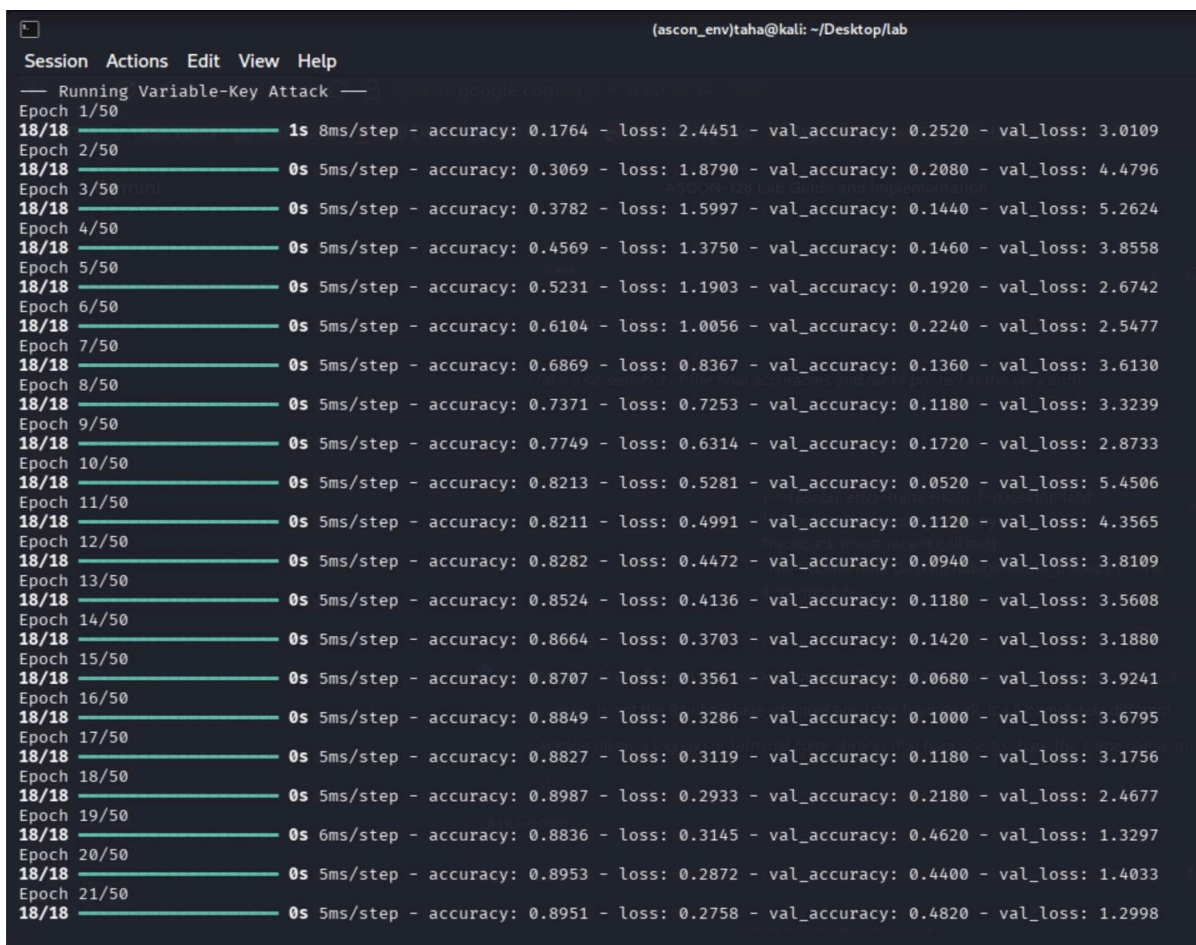


Figure 9: Variable-key training log (epochs 1–21) showing accuracy rising from 17.64% to 89.51%; validation accuracy fluctuates due to key diversity in the dataset

Screenshot: Variable-Key Final Results (SS-10)

```
Epoch 45/50
18/18 — 0s 5ms/step - accuracy: 0.9289 - loss: 0.1800 - val_accuracy: 0.4460 - val_loss: 1.9571
Epoch 46/50
18/18 — 0s 5ms/step - accuracy: 0.9313 - loss: 0.1859 - val_accuracy: 0.2640 - val_loss: 2.8672
Epoch 47/50
18/18 — 0s 5ms/step - accuracy: 0.9262 - loss: 0.1829 - val_accuracy: 0.9380 - val_loss: 0.1816
Epoch 48/50
18/18 — 0s 5ms/step - accuracy: 0.9360 - loss: 0.1760 - val_accuracy: 0.8080 - val_loss: 0.5090
Epoch 49/50
18/18 — 0s 5ms/step - accuracy: 0.9351 - loss: 0.1718 - val_accuracy: 0.7400 - val_loss: 0.7504
Epoch 50/50
18/18 — 0s 6ms/step - accuracy: 0.9409 - loss: 0.1545 - val_accuracy: 0.8740 - val_loss: 0.3957

Variable-Key Attack Accuracy: 87.30%
Variable-Key Average Key Rank: 1.00
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This
command using instead the native Keras format, e.g. `model.save('my_model.keras')` or `keras.saving.save_model(model
Fixed-Key Model on Variable-Key Data: 37.20%

(ascon_env)-(taha@kali)-[~/Desktop/Lab]
$
```

Figure 10: Variable-Key Attack Accuracy 87.30%, Average Key Rank 1.00; cross-evaluation of fixed-key model on variable-key data yields only 37.20%, confirming key-specific overfitting

Variable-Key Training Curves

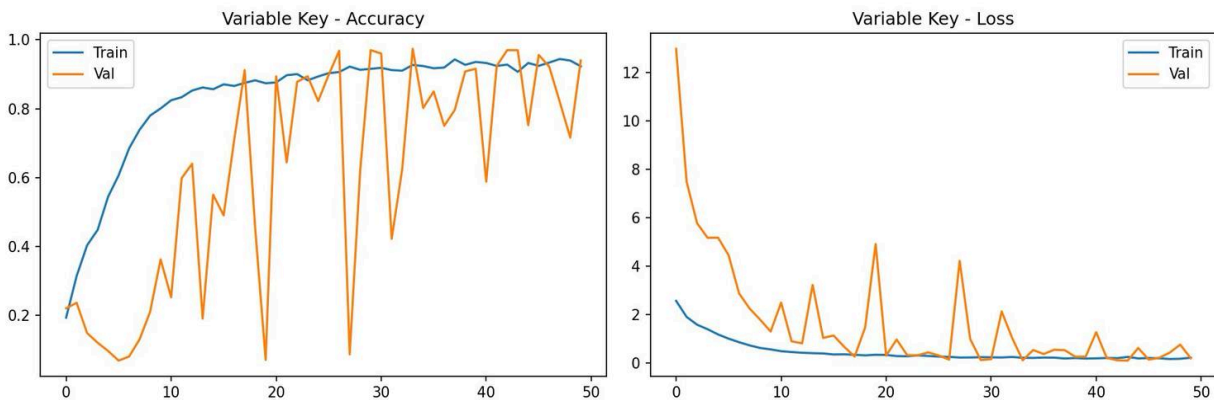


Figure: Variable Key — Accuracy & Loss curves over 50 epochs. Training accuracy (blue) reaches ~94% while validation accuracy (orange) is more erratic due to key diversity, yet ultimately stabilizes around 87–92%, demonstrating effective generalization across unseen keys.

Analysis & Security Insight

Key Observations

- ASCON-128, despite its NIST-standardized theoretical security, remains vulnerable to Hamming Weight-based power side-channel attacks when implemented without countermeasures.
- The fixed-key attack achieved 93.9% accuracy with an average key rank of 1.90, confirming that deep learning models can reliably exploit static leakage patterns.
- The variable-key model achieved superior generalization (key rank 1.00) by learning Hamming weight features that are key-independent, aligning with the power model assumption.
- The cross-evaluation result (37.2%) highlights a critical limitation: models trained on fixed keys fail to generalize — consistent with findings in the SCA literature for profiled attacks.

Security Implications

These results underscore that even NIST-standardized algorithms are not automatically secure against physical attacks. Any deployment of ASCON on embedded hardware must incorporate countermeasures such as:

- Masking: randomizing intermediate values to decorrelate power from sensitive data
- Hiding: randomizing execution time or adding dummy operations
- Algorithmic noise injection to reduce SNR of captured traces

Without such protections, an adversary with physical access and a small number of power traces can recover the secret key with high confidence.

Grading Rubric

Assessment Area	CLO / PLO	Does Not Meet (0–1)	Meets (2–3)	Exceeds (4–5)	Score
Lab Preparation & Environment Setup	CLO 3 GA-5 (P-3)	Fails to set up required environment; tools missing or non-functional.	Correctly configures environment with minor issues; tools mostly functional.	Confident, independent setup; all tools installed and verified prior to lab.	/ 2
Execution of Lab Procedures & Tool Usage	CLO 3 GA-5 (P-3)	Fails to execute required steps; solution does not run correctly.	Successfully executes core procedures; tools used adequately.	Correct, precise execution of all lab steps with full command mastery.	/ 3
Observations, Data Recording & Screenshots	CLO 3 GA-5 (P-3)	Little to no recorded observations; screenshots absent or irrelevant.	Records most observations with adequate screenshots; minor omissions.	Thorough, well-annotated documentation; all observations clearly linked to objectives.	/ 2
Analysis & Security Insight	CLO 3 GA-5 (P-3)	No meaningful analysis; fails to identify vulnerabilities.	Identifies key findings with reasonable analysis; connects to objectives.	Insightful, well-reasoned analysis; correctly interprets security implications.	/ 2
Lab Report Quality & Submission	CLO 3 GA-5 (P-3)	Poorly structured or not submitted; sections missing; unclear writing.	Complete and adequately organized; minor formatting issues.	Professionally written; all sections complete; follows SEECs guidelines.	/ 1